# Smart Medical Diagnostics & Risk Scoring System

| | |
|---|---|
| **Team** | **G4** |
| **Team members** | **1 Fady Talaat Edwar Henien**<br>**2 Mennatullah Abdul Monem Samy Abdul Monem**<br>**3 Shahd Ashraf Hassan**<br>**4 Mennatullah Mohammed Ali**<br>**5 Hossam Ramzi Mohammed Elbohoty** |
| **Supervisor** | **Zeyad Ayman** |
| **Repository** | |

# Contents

**Contents** ..... 2

**1. Project Planning & Management** ..... 3

1. Project proposal ..... 3
2. Project plan ..... 4
3. Task assignment & roles ..... 5
4. Risk assessment & mitigation ..... 6
5. Key performance indicators ..... 6

**2. Literature Review** ..... 7

1. Triage and why delays matter ..... 7
2. Early warning scores ..... 7
3. Machine learning at triage ..... 7
4. The MIMIC data ..... 8
5. What this means for our project ..... 8
6. Feedback, evaluation and suggested improvements ..... 8

References ..... 8

**3. Requirements Gathering** ..... 10

1. Stakeholder analysis ..... 10
2. User stories and use cases ..... 10
3. Functional requirements ..... 11
4. Non-functional requirements ..... 12

# 1. Project Planning & Management

## 1. Project proposal

Our project is a smart diagnostics and risk scoring system for an emergency department (ED). Patients arrive in waves, and during busy hours the people at triage have only a few minutes for each one. A patient who looks stable but is not can end up waiting too long. We want to give triage staff a quick second opinion: a risk score and a High, Medium or Low flag for every waiting patient, shown in a Power BI dashboard.

The work is built on MIMIC data (ED, hospital and ICU tables) for 100 patients. A Python pipeline written with object-oriented design reads and checks the data, cleans it, builds features and passes it to a classification model. The results and the logs go into a SQL Server warehouse, and Power BI reads from there.

### The problem

Emergency triage units are slow to spot high-risk patients when many people arrive at once. The delay pushes back the medical intervention that those patients need. The information that could help (vital signs, lab results, earlier diagnoses) is usually there, but it is spread over several systems and nobody has time to read all of it during a rush.

### Objectives

1. Build one Python pipeline that takes the raw MIMIC files to a clean, scored dataset, and protect sensitive patient fields through encapsulation.
2. Deal with missing and invalid clinical values through custom exceptions and structured logs, so that nothing is dropped or changed silently.
3. Create a dynamic risk score and a High / Medium / Low class for each ED stay, and train a classification model (Random Forest) whose results can be read from a confusion matrix.
4. Store the clean data, the scores and the logs in a SQL Server warehouse.
5. Deliver a Power BI suite with three views: Triage, Patient and Executive.

### Scope

| In scope | Out of scope |
|---|---|
| ED triage records, vital signs and ED diagnoses<br>Lab results, admissions and diagnosis history from the hospital module<br>ICU stays, used as outcome information<br>A rule-based risk score and a Random Forest classifier<br>SQL Server warehouse with staging, clean and output tables<br>Power BI suite with the three views<br>Documentation and code on GitHub | A connection to a live hospital system (we replay historical ED arrivals in time order to imitate live intake)<br>Real clinical decisions: the score supports triage staff and does not replace them<br>Treatment recommendations<br>Any data beyond the 100-patient extract<br>Applications outside Power BI (mobile or web apps) |

*Table 1. Project scope.*

## 1.2 Project plan

The plan follows the DEPI schedule (the right-hand column of the program guideline). Table 2 shows the official dates and Figure 1 shows how our tasks sit between them.

| Deliverable | DEPI due date | What we hand in |
|---|---|---|
| **Project planning & management** | 16 Oct 2026 | Proposal, plan, roles, risks, KPIs |
| **Literature review** | 16 Oct 2026 | Related work, feedback and suggested improvements |
| **Requirements gathering** | 16 Oct 2026 | Stakeholders, user stories, functional and non-functional requirements |
| **System analysis & design** | 6 Nov 2026 | Diagrams, database design, UI mock-ups, technology stack |
| **Implementation (source code & execution)** | 30 Nov 2026 | Working pipeline, warehouse, dashboards, README |
| **Final presentation, testing & reports** | 4 Dec 2026 | Test report, user manual, technical documentation, presentation |

*Table 2. DEPI deliverables and due dates.*

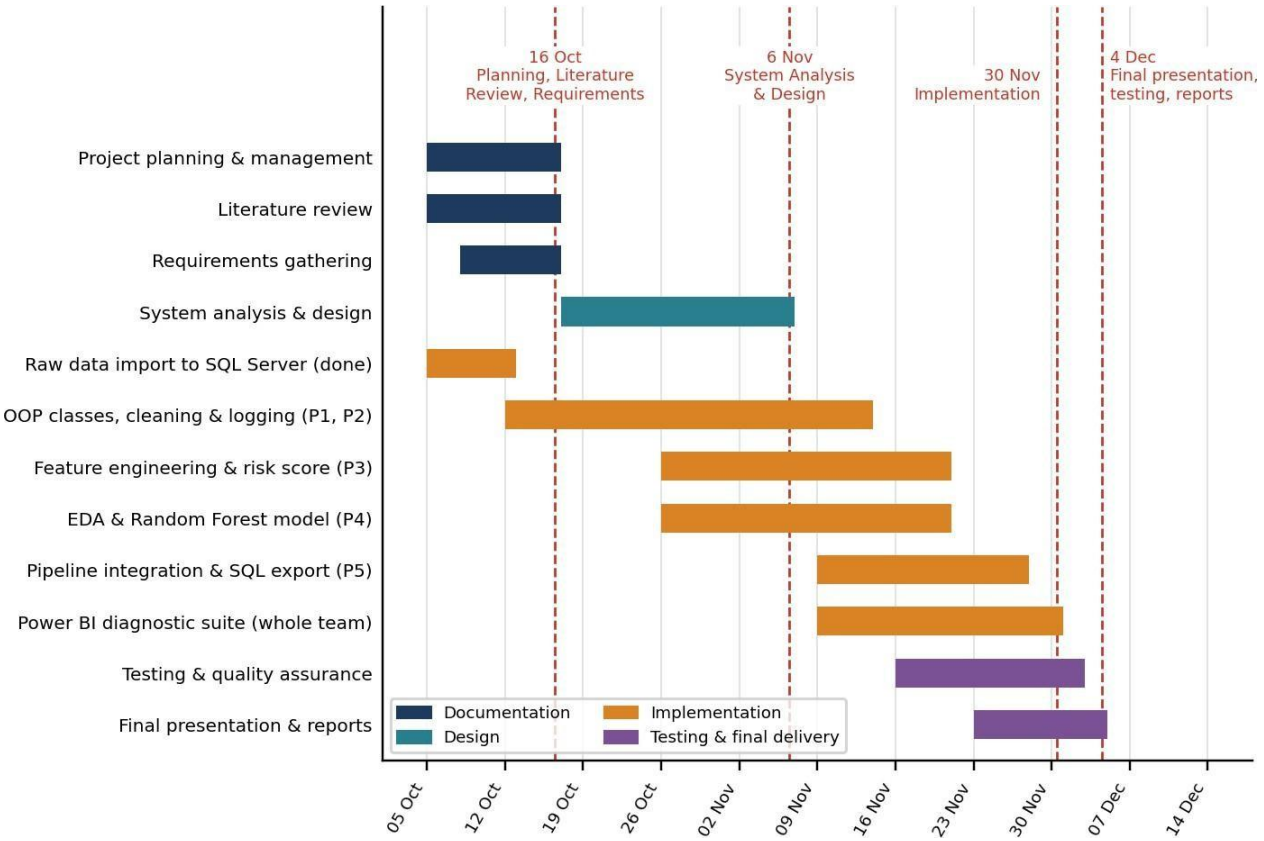



*Figure 1. Project timeline. Red dashed lines mark the DEPI due dates.*

Implementation starts before the design deadline on purpose. The raw data is already imported into SQL Server, and the four code modules (classes, cleaning, features and scoring, model) can be written side by side as long as everybody agrees on what each one takes in and hands back. The integration work comes last because it needs the other modules to exist. Testing overlaps with the end of implementation so that bugs are found while there is still time to fix them.

### Resources

- People: five team members, each owning one part of the pipeline (section 1.3).
- Software: Python 3, a local SQL Server 2022 instance, Power BI Desktop, GitHub for version control.
- Data: 37 MIMIC CSV files in three folders (ed, hosp, icu), about 135 MB in total.

## 1.3 Task assignment & roles

Each person owns one stage of the pipeline. The Power BI suite is shared, because every stage feeds it.

| Role | Responsibility | Main output |
|---|---|---|
| **Person 1 OOP & security** | Designs the core classes (Patient, TriageRecord) and applies encapsulation so that sensitive fields are private and reached only through controlled methods. | Core class module |
| **Person 2 Cleaning & reliability** | Handles missing values and outliers, writes the custom exceptions for missing or invalid clinical data, and sets up structured logging. | Cleaning module, exception classes, log files |
| **Person 3 Features & risk score** | Builds features from vitals, age and history, and defines the dynamic risk score with its High / Medium / Low bands. | Feature module, scoring rules |
| **Person 4 EDA & model** | Explores the data, trains the Random Forest classifier and evaluates it with a confusion matrix and per-class results. | EDA findings, trained model, evaluation report |
| **Person 5 Integration & warehouse** | Merges the four modules into one main pipeline, writes the SQL warehouse export, and tests the whole pipeline from start to finish. | Main pipeline, warehouse tables, integration test results |
| **Whole team** | Power BI diagnostic suite, documentation, final presentation. | Dashboards, documents |

*Table 3. Roles and responsibilities.*

## 1.4 Risk assessment & mitigation

These are the risks we think matter most. Several come directly from what we found when we went through the data (section 3.5).

| ID | Risk | Likelihood | Impact | How we deal with it |
|---|---|---|---|---|
| R1 | The dataset is small (100 patients, 222 ED stays, 15 in-hospital deaths), so the model may be unstable. | High | High | Split train and test data by patient, use cross-validation, keep the model simple, and compare it with a plain rule-based score. |
| R2 | Leakage: some fields reveal the outcome. For example, every admitted ED stay has an admission id. | Medium | High | Agree a list of allowed features (triage-time information only) and keep fields such as disposition and admission id out of the model. |
| R3 | Dirty clinical values: a temperature in Celsius, a diastolic pressure of 879, free-text pain scores, 10 to 12% of vitals missing. | High | Medium | Range checks, custom exceptions, documented cleaning rules, and a log entry for every change. |
| R4 | Integration problems when four people's code is merged. | Medium | High | Agree the input and output of each module early, merge in small steps, and test the combined pipeline on a small sample first. |
| R5 | The raw tables were imported as text, so type conversion errors and trailing spaces can break joins. | Medium | Medium | Trim and convert types in a separate clean layer, and compare row counts with the source files. |
| R6 | Deadlines are close together (6 Nov, 30 Nov, 4 Dec). | Medium | High | Weekly check-ins, core pipeline first and extras later, documentation written along the way. |
| R7 | Patient data is shared by mistake (for example pushed to GitHub). | Low | High | Only code and documents go to the repository, data files stay local, and every push is checked. |
| R8 | Power BI is not truly real-time when the source is historical data. | High | Low | Replay ED arrivals in time order to imitate live intake, and say so clearly in the report and the demo. |
| R9 | A team member is unavailable for a period. | Low | Medium | Every module has short handover notes so that another person can take over. |

*Table 4. Risk register.*

## 1.5 Key performance indicators

The targets below are goals, not results. Model targets in particular are provisional until a first baseline run exists; with so few records we will review them rather than chase a number.

| KPI | How we measure it | Target |
|---|---|---|
| **Data load completeness** | Source files loaded into staging, and row counts compared with the CSV files | 37 of 37 files, counts match |
| **Pipeline reliability** | Full run from raw files to warehouse without manual steps; failed records appear in the log | No unlogged rejections |
| **Data quality** | Out-of-range clinical values remaining in the clean tables | 0 |
| **Sensitivity for high-risk patients** | Recall of the High-risk class on held-out patients (confusion matrix) | At least 0.80, to be reviewed after the baseline |
| **Agreement with triage level** | Share of patients with ESI acuity 1 who are placed in the High class | At least 90% |
| **Dashboard speed** | Time for the Triage View to load or refresh on a team laptop | Under 5 seconds |
| **Usability** | Test users who find the highest-risk patient on the Triage View within 10 seconds | At least 4 out of 5 |
| **Documentation on time** | Deliverables on GitHub by the DEPI dates | 6 of 6 |

*Table 5. Key performance indicators.*

# 2. Literature Review

This section covers the ideas and earlier work our project builds on: how triage is done today, how early warning scores work, what machine learning has been used for in the emergency department, and what the MIMIC data offers.

## 1. Triage and why delays matter

Most emergency departments sort patients with a structured triage scale. One of the most widely used is the Emergency Severity Index (ESI), a five-level scale where level 1 is the most urgent, a patient who needs an immediate life-saving intervention, and level 5 the least urgent (Gilboy et al., 2012). The triage nurse assigns the level after a short assessment that looks at vital signs, the main complaint and how many resources the patient is likely to need. This is the acuity value stored in the MIMIC ED triage table. In our extract it takes the values 1 to 4, and 15 of the 222 triage records have no acuity at all.

The scale is simple and widely understood, but it depends on one person's judgment made in a few minutes. When the department is full, that judgment is made under pressure. That is the situation our project is aimed at.

## 2. Early warning scores

Early warning scores take a different approach and turn a handful of routinely measured values into one number. The National Early Warning Score (NEWS) used in the United Kingdom combines respiratory rate, oxygen saturation, systolic blood pressure, pulse, temperature, level of consciousness and whether the patient is receiving extra oxygen. The higher the total, the sooner a clinician should respond (Royal College of Physicians, 2012).

Two things are useful for us. The score is simple enough to calculate at the bedside, and it uses the same vital signs that sit in our triage table. The weakness is that the thresholds are fixed. They do not change with the patient's age or history and they do not learn from data. Our risk score keeps the readability of a rule-based score but is meant to adapt, and we will use a NEWS-style rule as a baseline when we judge the model.

## 3. Machine learning at triage

Several studies have tried to predict outcomes at the moment of triage using machine learning. Hong, Haimovich and Taylor (2018) predicted hospital admission from triage information and found that adding the patient's earlier history improved the predictions. Raita et al. (2019) compared machine learning models with a conventional triage-based reference model for outcomes such as intensive care admission and in-hospital death, and the machine learning models did better.

Both studies use far more data than we have, so we cannot expect the same numbers. What we take from them is the design: use information that is available at triage, add history where it helps, and judge the model on outcomes that matter clinically instead of looking only at accuracy.

We chose a Random Forest as the classifier. It builds many decision trees on random samples of the data and random subsets of the variables, and combines their votes (Breiman, 2001). It copes with a mix of numeric and categorical inputs, gives a reasonable first result without much tuning, and reports which variables were important, which helps when we explain the model to non-technical readers. With only 222 ED stays, a simple and stable model suits us better than a more complex one.

## 4. The MIMIC data

MIMIC-IV is a freely available collection of de-identified patient records from Beth Israel Deaconess Medical Center in Boston (Johnson et al., 2023a). The emergency department data are released as a separate module, MIMIC-IV-ED (Johnson et al., 2023b), next to the hospital and ICU modules.

Our team works with a 100-patient extract. It has 37 CSV files in three folders, about 135 MB in total. The 222 ED stays belong to 64 of the 100 patients, and the extract also holds 275 hospital admissions and 140 ICU stays. All dates were shifted into the 2100s by the database creators to protect privacy, so only the time between two events means anything. Access to the full MIMIC-IV requires credentialing and a data use agreement, which is a good reminder that these records need careful handling even though names and direct identifiers have been removed.

## 5. What this means for our project

- Model features come from information available at triage, plus patient history. Anything recorded after triage (for example the ED disposition) stays out.
- Vital signs, age and history are combined in one score instead of looking at each vital sign alone.
- We judge the model by how well it finds the High-risk patients (recall and the confusion matrix), not only by overall accuracy.
- We keep the model small and explainable, and keep a rule-based score next to it so staff can see why a patient was flagged.

## 6. Feedback, evaluation and suggested improvements

The table below is for the supervisor's assessment. It will be filled in after the review and the team's response added next to each point.

| Date | Reviewer | Feedback | How we responded |
|---|---|---|---|
| | | | |
| | | | |
| | | | |

*Table 6. Supervisor feedback log (to be completed after review).*

Table 6. Supervisor feedback log (to be completed after review).

Our own suggestions for improving the project if time allows:

- Repeat the evaluation on a larger MIMIC sample if access to the full database is approved.
- Show, for each flagged patient, which values pushed the score up, so staff can check the reasoning.
- Add waiting time to the Triage View, if the timestamps in the data support it.
- Add a simple forecast of arrivals per hour to support peak-load planning in the Executive View.

The final grading criteria are set by the program. We will add them here once we have the official breakdown.

## References

Breiman, L. (2001). Random forests. Machine Learning, 45(1), 5-32.

Gilboy, N., Tanabe, P., Travers, D., & Rosenau, A. M. (2012). Emergency Severity Index (ESI): A triage tool for emergency department care, version 4. Implementation handbook, 2012 edition. AHRQ Publication No. 12-0014. Agency for Healthcare Research and Quality.

Hong, W. S., Haimovich, A. D., & Taylor, R. A. (2018). Predicting hospital admission at emergency department triage using machine learning. PLoS ONE, 13(7), e0201016.

Johnson, A. E. W., Bulgarelli, L., Shen, L., et al. (2023a). MIMIC-IV, a freely accessible electronic health record dataset. Scientific Data, 10, 1.

Johnson, A., Bulgarelli, L., Pollard, T., Celi, L. A., Mark, R., & Horng, S. (2023b). MIMIC-IV-ED (version 2.2). PhysioNet.

Raita, Y., Goto, T., Faridi, M. K., Brown, D. F. M., Camargo, C. A., & Hasegawa, K. (2019). Emergency department triage prediction of clinical outcomes using machine learning models. Critical Care, 23, 64.

Royal College of Physicians. (2012). National Early Warning Score (NEWS): Standardising the assessment of acute-illness severity in the NHS. London: RCP.

# 3. Requirements Gathering

The requirements come from three places: the project brief, the structure of the MIMIC data, and published descriptions of how triage works. They should be reviewed with the supervisor and, if possible, with someone who works in an emergency department.

## 1. Stakeholder analysis

| Stakeholder | What they need | How they relate to the system |
|---|---|---|
| **Triage nurse** | See quickly who needs attention first, with a flag that is clear at a glance | Main user of the Triage View |
| **ED physician** | See why a patient is flagged, and read lab and diagnosis history | Main user of the Patient View |
| **ED manager / hospital executive** | Understand throughput, busy hours and resource use | Main user of the Executive View |
| **Data team (our team)** | A pipeline that runs from start to finish, leaves a log, and gives the same result on a rerun | Builds and maintains the system |
| **Hospital IT / data governance** | Patient data protected and access controlled | Sets the security and privacy limits |
| **Patients** | Faster attention when they are seriously ill, and privacy | Indirect: they benefit from earlier care |
| **DEPI supervisor / evaluators** | A documented, working and tested project | Assess the project |

*Table 7. Stakeholders.*

## 3.2 User stories and use cases

| ID | As a... | I want to... | So that... |
|---|---|---|---|
| US-01 | triage nurse | see waiting patients sorted by risk | the sickest patients are seen first |
| US-02 | triage nurse | see a High / Medium / Low flag written as text as well as color | I can read it quickly, even if I cannot tell the colors apart |
| US-03 | ED physician | see the vital signs and history that went into a score | I can judge whether the flag makes sense |
| US-04 | ED physician | see a patient's lab results over time | I can notice a trend |
| US-05 | ED physician | see earlier diagnoses, ED visits and ICU stays | I understand the patient's background |
| US-06 | ED manager | see arrivals by hour and by day of the week | I can plan staffing for busy periods |
| US-07 | hospital executive | see length of stay, admission rate and ICU transfers | I can judge how well the department is coping |
| US-08 | data team member | run the whole pipeline in one go and read the log | I know which records were rejected and why |
| US-09 | data steward | have sensitive fields hidden behind controlled methods | they are not exposed by mistake |

*Table 8. User stories.*

| Use case | Actor and precondition | Main flow | Alternative flow and result |
|---|---|---|---|
| **UC-01 View the triage priority queue** | Triage nurse. The pipeline has run and the scores are in the warehouse. | 1. Open the Triage View.<br>2. The queue lists patients by risk score.<br>3. Filter by risk class or arrival mode.<br>4. Select a row to see more. | If a patient has no score because data was missing, the row is marked "not scored" and the case is in the log.<br>Result: the sickest patients are at the top. |
| **UC-02 Review patient history** | ED physician. A patient is selected, or opened from the queue. | 1. Open the Patient View.<br>2. Read the lab trend.<br>3. Check the visit timeline and diagnoses. | If a patient has no lab results, the chart shows an empty state with a message.<br>Result: the physician sees how the patient has progressed. |
| **UC-03 Run the data pipeline** | Data team member. CSV files and SQL Server are available. | 1. Start the pipeline.<br>2. Load and validate the data.<br>3. Clean it, build features, score.<br>4. Export to the warehouse.<br>5. Check the log. | If a record is invalid, an exception is raised and logged, and a rule imputes, flags or excludes it. The run continues.<br>Result: the warehouse is updated. |

*Table 9. Main use cases.*

## 3.3 Functional requirements

| ID | Requirement | Priority |
|---|---|---|
| FR-01 | The pipeline reads the ED, hospital and ICU CSV files from the project folder. | Must |
| FR-02 | Raw data is loaded unchanged into staging tables in SQL Server, and the row counts are compared with the source files. | Must |
| FR-03 | Every triage record is checked for required fields and plausible ranges (heart rate, blood pressure, oxygen saturation, temperature, pain). | Must |
| FR-04 | When a value is missing or invalid, the pipeline raises a custom exception that names the field and the record. | Must |
| FR-05 | Every exception and major pipeline event is written to a structured log with time, stage, record id and reason. | Must |
| FR-06 | Cleaning trims text, fixes unit mistakes (for example Celsius entered in a Fahrenheit field), standardizes labels and converts pain scores. | Must |
| FR-07 | Outliers are handled by documented rules and are never deleted silently. | Must |
| FR-08 | Patient and TriageRecord objects keep sensitive fields private and give access only through controlled methods. | Must |
| FR-09 | Features use only information available at triage time, plus patient history. | Must |
| FR-10 | Each ED stay gets a dynamic risk score and a High, Medium or Low class. | Must |
| FR-11 | A Random Forest classifier is trained and evaluated with a confusion matrix and per-class results. | Must |
| FR-12 | Clean data, risk scores and logs are exported to the SQL Server warehouse. | Must |
| FR-13 | Triage View: a priority queue sorted by risk, with color-coded flags. | Must |
| FR-14 | Patient View: lab history and diagnostic progression over time for one patient. | Must |
| FR-15 | Executive View: throughput, peak load and resource use. | Must |
| FR-16 | Filters (period, arrival mode, risk class) on every view. | Should |
| FR-17 | The whole pipeline runs from a single entry point and gives the same results when it is run again. | Should |

*Table 10. Functional requirements.*

## 3.4 Non-functional requirements

| Category | Requirement | How we will check it |
|---|---|---|
| **Performance** | The full pipeline (all 37 files) finishes in under 30 minutes on a team laptop. The Triage View refreshes in under 5 seconds. | Timed test runs |
| **Security and privacy** | Sensitive attributes are private inside the classes. No patient data or passwords in the GitHub repository. SQL Server is accessed with Windows authentication. | Code review, repository check before each push |
| **Usability** | Risk class can be read without relying on color. Each view fits on one screen. A patient's history is reachable from the queue in two clicks. | Walkthrough with test users |
| **Reliability** | One bad record never stops the run. Failed records appear in the log. Reruns do not create duplicate rows. | Integration tests with deliberately broken records |
| **Accuracy and traceability** | Every score can be traced to its source record and to the model version that produced it. | Inspect risk_scores and pipeline_log |
| **Maintainability** | Code is modular, follows one naming convention and is documented. | Peer review |
| **Portability** | The project runs on Windows with Python 3 and SQL Server 2022 by following the README. | Fresh install by a team member |

*Table 11. Non-functional requirements.*